Blu Visual Teacher Hackathon — Problem Statement

Oct 28, 2026 ·

In 24 hours, build an offline system that turns a typed concept into a short, narrated, animated lesson in English and at least two Indian languages.

Why this matters

Concepts like a loop in code, a chemical reaction or a law of physics are easier to grasp when you see them move step by step and hear them explained in your own language. Today, one short explainer video like that takes a teacher hours of scripting, drawing, recording and editing.

Your job is to build **Blu Visual Teacher**: a system that makes that video in minutes, on a laptop, without the internet.

Your challenge

Build a local-first engine that takes a concept (for example, "How does a Python loop work?") and produces a 30–90 second visual lesson: animated scenes, on-screen captions and a voice-over in the learner's chosen language — English plus at least two Indian languages such as Hindi, Tamil, Telugu, Bengali or Marathi. The lesson must play in the browser and export as an MP4.

You are building a *video engine*, not a video-generating AI model. A small local language model writes the lesson plan as structured data. Your own code turns that data into animated diagrams, code walkthroughs and narration. Correct, clear teaching matters more than cinematic visuals.

Ground rules

These are hard rules. Breaking one makes your team ineligible for the main prize.

- **No external AI APIs.** No OpenAI, Gemini, Claude, ElevenLabs, Google Translate, or Google or Microsoft cloud voices. Every model runs on your own machine.
- **Laptops only.** Plan for a CPU-only laptop with 8–16 GB of RAM. A GPU can speed things up, but your demo must also work without one.
- **Works offline.** Once models and packages are downloaded, the whole pipeline must run with the internet switched off.

- **Data, not code.** The language model returns JSON only. Your app never runs JavaScript or Python that the model wrote. Visuals come only from components you built.
- **Light for students.** The finished lesson must play smoothly in a browser on a modest phone.
- **Licences checked.** Every model, voice and asset you use must allow this use. List them in your README.

What you must build

Your system takes a learner from a typed concept to a narrated lesson in five stages. All five are needed for a full score.

#	Stage	What it must do	Done when
1	Concept input	A simple page where the learner types a topic and picks a level (beginner / intermediate), a language and a target length	Topic, level and language reach your backend
2	Lesson planner (local LLM)	A local model (for example, via Ollama) writes a lesson of 3–6 scenes as JSON: title, scene type, on-screen content, narration text	Output passes your schema check; a bad response falls back to a built-in sample lesson so your demo never breaks
3	Visual engine	Reusable components (for example, React + SVG + CSS animation) for at least three scene types: title card, diagram with labels and arrows, and code walkthrough	Plays a lesson with play, pause, next scene and replay
4	Multilingual narration	Narration and captions in at least 3 languages — English with an Indian accent plus 2 Indian languages — all from local models; captions in the right script (Devanagari, Tamil, Telugu and so on)	The same lesson plays in all 3 languages, with audio, captions and scenes in sync

#	Stage	What it must do	Done when
5	MP4 export	Combine the rendered scenes, voice-over and captions into a downloadable video (for example, with FFmpeg)	A playable 720p MP4 of the full lesson

Key design rule: the voice-over sets each scene's length. Generate the narration audio first, then make each scene last as long as its audio plus a short pause. If you hard-code scene durations, your audio and visuals will drift apart.

Language tip: small local models write weaker Hindi or Tamil than English. A reliable approach is to plan the lesson in English, then translate the narration and captions with a local translation model (for example, AI4Bharat's IndicTrans2). Keep code, numbers and technical terms in English where students would say them that way.

Input and output

Input: a topic (free text), a learner level, a language code (for example en, hi, ta, te, bn, mr) and a target length in seconds.

Lesson plan (what your LLM returns): JSON that matches a schema you define and validate. Use the example below as a starting point; you may add scene types and fields.

```
{
  "title": "How a Python loop works",
  "level": "beginner",
  "language": "hi",
  "scenes": [
    {
      "type": "title_card",
      "heading": "How does a Python loop work?",
      "narration": "A loop repeats an instruction for each value in a
sequence."
    },
    {
      "type": "code_walkthrough",
      "heading": "Step 1: Read the code",
      "code": "for i in range(3):\n    print(i)",
      "highlight_lines": [1],
      "narration": "range of three gives three values: zero, one and two."
    },
    {
      "type": "diagram",
      "heading": "Step 2: Watch each iteration",
```

```
      "items": ["i = 0", "i = 1", "i = 2"],
      "narration": "Each pass takes the next value, then runs print i."
    }
  ]
}
```

Scene length is **not** in the plan: it comes from the narration audio (see the key design rule above).

Output: the lesson playing in the browser, plus a downloadable MP4 (1280×720, H.264 video, AAC audio) with captions in the lesson's own script, burned in or as a separate .srt file.

Reference video: at kickoff we will play a 29-second sample lesson on Python loops. It shows the look we expect: step headings, a caption bar and a progress bar.

Bonus points

Bonus points count only once all five core stages work.

- **More languages:** each Indian language beyond the required two, and a switch that re-voices a finished lesson in another language without planning it again.
- **Voice choice:** a male or female voice, and a speed control.
- **More scene types:** a graph or chart, a timeline, a labelled illustration, a quiz question at the end.
- **Edit before render:** let a teacher change a scene's narration or content, then regenerate only that scene.
- **Reuse:** cache finished lessons by topic, language and version so the same lesson is never generated twice.
- **Speed:** show how long each stage takes on your weakest laptop.

What to submit

Submit by the end of hour 24:

1. **Code repository** with a README covering setup steps, the models and voices you used and their licences, and the laptop specs you tested on.
2. **Two generated MP4 lessons**: one on a topic the judges give you on the day, and one on a topic of your choice. Each must be in English and in one Indian language.
3. **A 5-minute live demo**: type a new topic and language the judges call out, generate the lesson live, and play it. Then 3 minutes of questions.

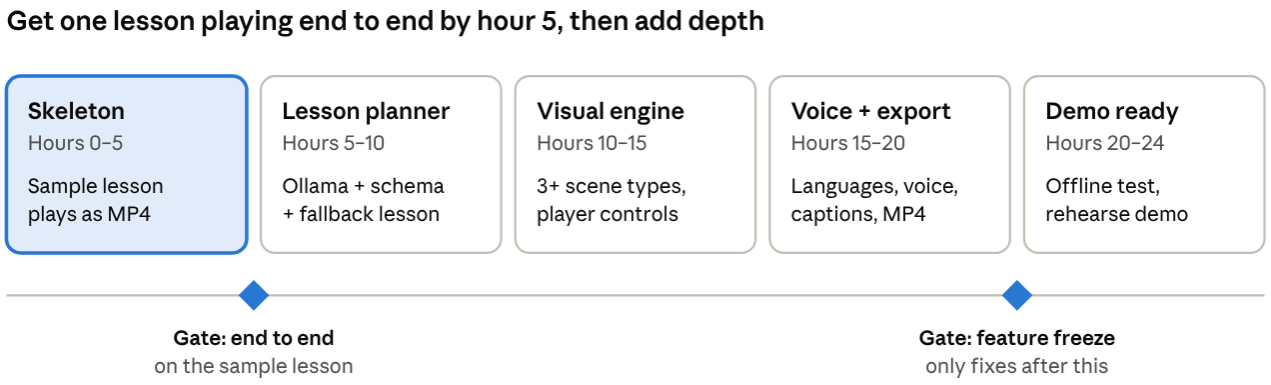
The live demo runs with Wi-Fi off. If a model call fails, your fallback lesson must play instead.

How you will be judged

Criterion	Weight	What judges look for
Works end to end	25%	Topic in, narrated MP4 out, live and offline, with no manual steps
Teaching quality	20%	Content is correct; scenes build understanding step by step; a beginner could follow it
Visual clarity	15%	Clean, readable scenes; animation that explains rather than decorates
Languages, narration and sync	15%	Natural voice in every language; accurate translation in the right script; audio, captions and visuals stay in step
Engineering and robustness	15%	Schema validation, fallback lesson, clean separation of planner, renderer and exporter
Speed and footprint	10%	Generation time and memory use on a modest laptop; smooth playback on a phone

Bonus features can add up to 10 extra points.

Suggested 24-hour plan



suggested 24-hour plan · 5 phases, 2 gates

This plan is a suggestion, not a rule. The gate that matters most is the first: a fixed sample lesson that plays end to end as an MP4. If time runs short, cut bonus features and extra scene types, never the end-to-end flow.

Before you arrive

Rules

- All code must be written during the 24 hours. Open-source libraries and pre-trained models are allowed.
- Download pre-trained models and packages **before** the event. Venue Wi-Fi may be slow, and training your own foundation model is out of scope.
- Open source does not always mean free for any use. Check each model's weights, code and voice licences separately.

Download

- [ ] Ollama, plus one small instruction model (around 3B parameters for 8–16 GB laptops)
- [ ] Local text-to-speech for every language you plan to support. Kokoro (`kokoro-onnx`) runs on CPU and covers Indian-accented English and Hindi (voices `hm_omega`, `hm_psi`, `hf_alpha`, `hf_beta`). For other Indian languages, try AI4Bharat's open Indic voice models (such as Indic Parler-TTS) and test their speed on your laptop.
- [ ] A local translation model for Indian languages, such as AI4Bharat's IndicTrans2
- [ ] Noto Sans fonts for each Indian script you support, so captions and the MP4 show text correctly
- [ ] FFmpeg
- [ ] Node.js and Python with your packages already installed (for example React, FastAPI, Playwright)
- [ ] A fixed sample lesson JSON to test your renderer before the model is connected

Our advice: get the whole chain working on one hard-coded lesson first, then connect the language model. A simple lesson that plays end to end beats a clever model with no video.